

Titans: Learning to Memorize at Test Time

Abanavas Stavros, Jheeta Lydia, Nikitaki Lamprini, Zangogianni Violeta
ECE, NTUA

June 2026

Abstract

After the groundbreaking 2017 paper "Attention is All you Need", Transformers monopolized the interest in the AI space, and became almost synonymous with modern LLMs. The innovation they brought was the Attention mechanism, which acts as a memory that updates the context of the tokens processed by the model by looking at and comparing surrounding tokens. However, the main drawback of Attention is that it does not scale easily with the size of the context we want the model to remember. As a result, information located at the beginning of the processed sentences is completely lost as the model moves forward in time.

Thus, Behrouz et al. in 2025 proposed a new architecture, the Titans, which extends previous models by adding a short-term memory that can efficiently store very large contexts in terms of memory and compute time, achieving better results.

In this paper, we will replicate the results shown in the aforementioned paper, albeit using smaller models. The main objective is to show why Titans are an improvement over Transformers for most tasks. We further report a *negative result* on a multi-hop reasoning task, which we were unable to reproduce under free-tier compute, and we analyze its root cause. Finally, we present a *positive result* on long-term time-series forecasting, where our small-scale reimplementation qualitatively reproduces the paper's central claim.

1 Introduction

1.1 Attention

To understand why Titans are useful we need to briefly discuss Attention and specifically why we cannot rely just on it, if we want to handle really large context sizes.

Attention is a mechanism that can automati-

cally update the semantic content of a token by looking at previous (and possibly subsequent) tokens. This is achieved by comparing the similarity of the value vector of the current token (v_i), with the query vector of all the others (q_j). We then use softmax to determine the tokens that are more closely related to the one we are examining, and use a transformation V to go back to the embedding space and retrieve the updated information of those surrounding tokens.

The following equation summarizes this process.

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (1)$$

The matrices Q and K have dimensions $N \times d_k$, where N is the size of the context we are looking at and d_k is the dimension of the attention space. Crucially, the operation QK^T results in an $N \times N$ matrix. Therefore, we conclude that Attention does not scale linearly, but rather quadratically. If we take into account the massive context sizes of modern models, this turns into a serious bottleneck.

1.2 Neural Memory

Titans, in addition to Attention, use a new memory module called Neural Memory. This is a separate, shallow FNN that is not pre-trained, but rather it is constantly being trained at test-time. Its purpose is to learn to map tokens to a specific value, meaning it has a learning function:

$$\ell(\mathcal{M}_{t-1}; x_t) = \|\mathcal{M}_{t-1}(\mathbf{k}_t) - \mathbf{v}_t\|_2^2 \quad (2)$$

We obtain $\mathbf{k}_t$ and $\mathbf{v}_t$ by multiplying the input $\mathbf{x}_t$ with two matrices, W_k and W_v , that are considered fixed for the memory module.

This Neural Memory is trained in an "inner" training loop that remains active at test-time. To update the memory weights, we use the following two equations:

$$\begin{aligned}\mathcal{M}_t &= (1 - \alpha_t)\mathcal{M}_{t-1} + S_t \\ S_t &= \eta_t S_{t-1} - \theta_t \nabla \ell(M_{t-1}; x_t),\end{aligned}\tag{3}$$

The term S_t is the "Surprise Factor" received by the memory at that moment. Intuitively, a large S_t means that the context of the sentence being processed has changed significantly. How much we take the previous surprises into account is determined by the value of η_t . The term θ_t is just the learning rate of the memory.

There is also the "forget factor" α_t . This is necessary so that the model does not overfill with previous useless information.

In addition to this Short-Term Memory, we also add a Persistent Memory to the model, that is just a fixed vector of task-specific information that we concatenate to the input.

1.3 Titans

Now what is left to discuss is how we will incorporate this memory module into the broader architecture of the model. The authors suggest three designs:

- Memory as a Context (MAC)
- Memory as a Gate (MAG)
- Memory as a Layer (MAL)

In the MAC case, the input vector is simply concatenated with the result of the Neural Memory and then is processed as usual. In MAG architecture the input vector is not changed but it goes through attention with the result of the Neural Memory. In that case, the memory acts like a mask of the actually useful information of the input, so it acts like a gate. Finally, in a MAL Titan we simply pass the input through the memory and, thus, a new vector is produced that acts as our new input.

The following figure shows the three memory types that act inside a Titan and illustrates what information the Short-Term Memory is holding.

2 Experiment I: Needle-in-a-Haystack

In this task, we feed the model with a very long text segment filled with useless information (this is the haystack). We then ask it to retrieve a very

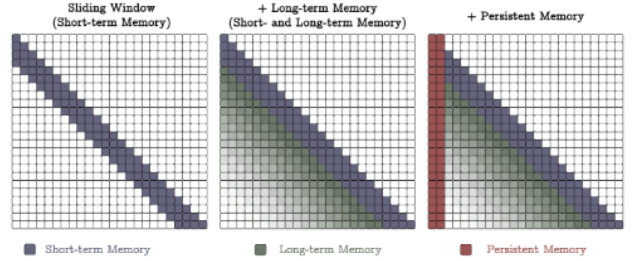


Figure 1: The types of memories that act in a Titan. We can see that the new Neural Memory (shown in green), is able to hold information from the hole sequence and not just the attention’s context

specific answer hidden somewhere inside the text (the needle).

We will test two models, one Baseline Transformer and one MAC Titan. They are of approximately the same size and they will be trained on exactly the same dataset (Wikitext). You can see the models’ in the following table.

Parameter	Baseline Transformer	MAC Titan
Dimension	256	≤256
Layers / Depth	4	4
Attention heads	8	8
Parameters	~12.6M	~12.6M
Training context	1,024 tokens	1,024 tokens
Vocabulary	GPT-2 (50,257)	GPT-2 (50,257)
Optimiser	AdamW, lr=3e-4	AdamW, lr=3e-4

Table 1: Model configuration for both architectures. Both are matched in parameter count and training data.

We experiment with different needle depths and multiple context sizes. In the following table we present the accuracy of both models.

Depth / Context	1K (in-dist)	2K	4K	8K	16K
<i>Baseline Transformer</i>					
10%	95	15	15	15	25
30%	100	15	15	20	10
50%	100	100	30	25	20
70%	100	100	20	35	25
90%	100	100	100	100	5
<i>MAC Titan</i>					
10%	100	100	100	20	20
30%	100	100	100	10	10
50%	100	100	100	35	25
70%	100	100	100	20	15
90%	100	100	100	25	15

Table 2: Retrieval accuracy (%) for each model, context length, and needle depth. A small depth percentage means that the needle is closer to the beginning of the test input.

The first thing that we notice is that when the

test context equals the model’s context (1K in this case), both models can perform exceptionally well. However, the Transformer’s weakness becomes apparent as the context gets larger and larger. When the needle gets out of the attention’s context (ie the needle is away from the end of the input segment) the model can no longer attend to it. This becomes more apparent in the case of 4K context, where the Transformer fails to find the needle except only when it is near the end (and thus inside the model’s context). On the contrary Titan’s Neural Memory can easily hold the necessary information for up to 4 times the trained context, thus making the model score perfectly.

To visualize the limitation of the Transformer, we show in the following figure the weights that the attention of the last token produced.

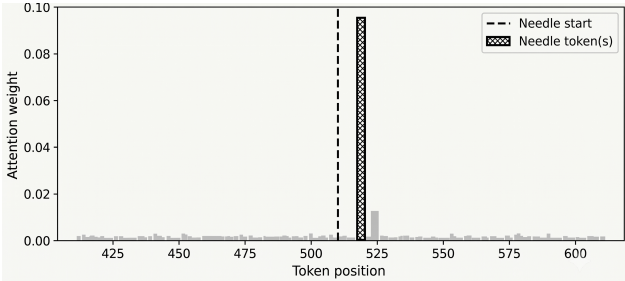


Figure 2: The weights of the attention of the last token

The way the Transformer finds the needle is totally based on the attention mechanism. Had the needle be placed outside of the model’s context, there would be no way for it to locate it.

What is also worth noting is that both models fail when the context gets too large (more than 4K tokens). This can be attributed firstly to the relative small scale of the models, but also to the fact that they were both trained on only 1K contexts and, as a result, none of the models learned to compress information for so much length. A different, but more computationally expensive, training involving bigger contexts would probably lead to a significantly improved performance of the Titan, but an underwhelming one for the Transformer.

3 Experiment II: In-Context Reasoning (A Negative Result)

We now organize our remaining findings around two experimental tracks. We first report a *negative result* on the reasoning task (Goal 2 of the paper), which we were unable to reproduce under

free-tier compute, and we analyze its root cause. We then report a *positive result* on the time-series forecasting task (Section 5.6 of the paper), where our small-scale reimplementation qualitatively reproduces the paper’s central claim. All experiments were run on a single Google Colab T4 GPU.

Experimental setup. For all comparisons we match the parameter budgets of the competing models so that differences in performance cannot be attributed to capacity. Unless stated otherwise, the Memory-as-Context (MAC) Titan and the long-term memory module (LMM) are taken from the `titans-pytorch` library, and the Transformer and LSTM baselines are implemented with identical patch/embedding pipelines. We report mean values over the evaluation grid; exact hyperparameters are listed in the Appendix.

Encouraged by the retrieval results, we next attempted to reproduce the more demanding reasoning experiment (Goal 2 of the original paper). Here a MAC Titan is expected to resist the “*Lost in the Middle*” effect and outperform a parameter-matched Transformer on *multi-hop* question answering over long, noisy contexts. Concretely, one or two supporting facts (in the style of the bAbI tasks) are inserted into WikiText-2 noise, and the model must combine them to answer a question placed at the end of the sequence. We evaluate a single-supporting-fact task (QA1) as a sanity check and a two-supporting-fact task (QA2) as the main multi-hop benchmark.

Across every iteration of this experiment, the MAC Titan failed to learn the task, while the Transformer baseline solved it. The baseline reached 95–100% accuracy across all context lengths, whereas the MAC Titan remained at chance level ($\approx 1/6 \approx 17\%$ for the six possible answer locations). The characteristic signature of this failure was that the MAC training loss plateaued at exactly $\ln(6) \approx 1.7918$ — the entropy of a uniform distribution over the six candidate answers — meaning the model learned only *that* the answer is one of the six location words, but never *which* one. Table 3 summarizes the outcome.

Model	QA1 (1K)	QA1 (4K)	QA2 (1K)	QA2 (4K)	Mean
Baseline Transformer	98	92	98	89	94
MAC Titan	14	16	17	11	15

Table 3: Reasoning accuracy (%). The MAC Titan stays at chance level ($\approx 1/6$) regardless of context length, while the Transformer baseline solves the task.

Two-Supporting-Fact QA (QA2): MAC Titan stays at chance — a negative result

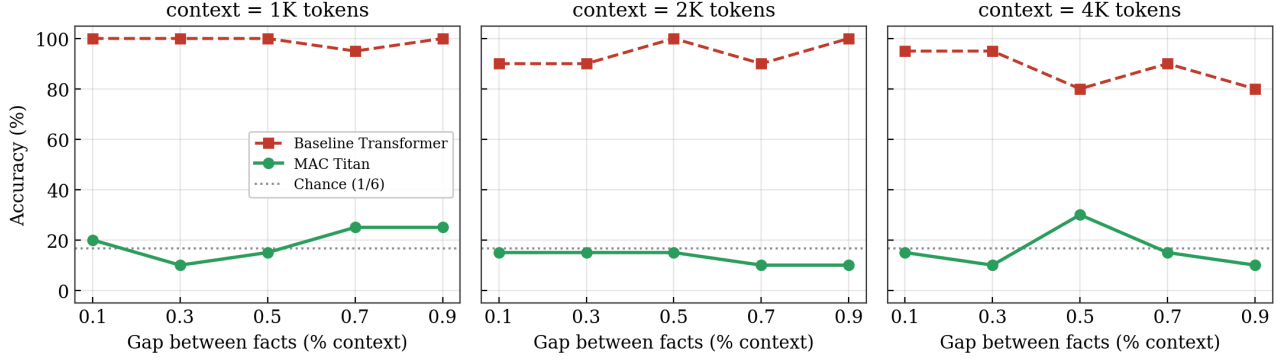


Figure 3: Accuracy on the two-supporting-fact task as a function of the gap between facts. The MAC Titan remains near chance across all gaps and context lengths.

Iterative debugging. We confirmed the failure was not caused by trivial implementation bugs by fixing, in turn: (i) a *data-leakage* issue in which the causal model could observe the answer token at its own input position, resolved by feeding `model(seq[:, :-1])` to both models; (ii) a *poisoned initialization* caused by transferring weights from a converged baseline, which trapped the MAC in the $\ln(6)$ minimum, resolved by training from scratch; (iii) a *learning-rate scheduler* miscount under gradient accumulation; and (iv) an overly hard 50,257-way objective, resolved with a restricted 6-way cross-entropy and a $128 \rightarrow 1024$ curriculum. None of these fixes lifted the MAC above chance on the long-context task.

Root cause. We attribute the failure to an architectural reality rather than a coding defect. The MAC architecture must learn cross-segment retrieval through its gradient-based inner loop, which requires pretraining on billions of tokens; the original paper uses 15–30B tokens and 170M–760M-parameter models. Our budget (≈ 2 M tokens, ≈ 5 K iterations) is roughly three orders of magnitude smaller, and no public pretrained Titans checkpoints are available. This conclusion is consistent with the independent study *Titans Revisited* (Di Nepi et al., 2025), which likewise could not reproduce the reasoning results without full-scale resources. The negative result is therefore itself informative: it delineates the regime in which the architecture can be reproduced under free-tier compute.

4 Experiment III: Long-Term Time-Series Forecasting

Guided by the root-cause analysis, we turned to the time-series forecasting experiment (Section 5.6 of the original paper), in which the Mamba module of the Simba framework is replaced by the standalone neural memory module (LMM). This task does not require learning cross-segment retrieval from scratch: the LMM compresses temporal structure directly into its weights, which is feasible at small scale. We evaluate on the **ETTh1** dataset (17,420 hourly records, 7 variables) using a channel-independent (CI) patching pipeline shared across all models, with an input length of 512 and prediction horizons of 96, 192, 336, and 720 hours. All models are matched in parameter budget. Table 4 reports the test-set MSE.

Model	96h	192h	336h	720h	Mean
LMM (Titans)	0.440	0.487	0.553	0.635	0.529
PatchTransformer	0.423	0.470	0.545	0.666	0.526
PatchLSTM	0.436	0.494	0.521	0.621	0.518
NaïveLast	1.705	1.780	1.832	1.986	1.826

Table 4: Test-set MSE on ETTh1 across prediction horizons (lower is better). The LMM trails the Transformer at short horizons but overtakes it at 720h.

Main finding. The LMM’s advantage grows monotonically with the prediction horizon. At short horizons the Transformer is marginally better (-4.0% at 96h, -3.6% at 192h), the two are essentially tied at 336h (-1.5%), and at the longest horizon the LMM *overtakes* the Transformer (0.635 vs. 0.666, a $+4.7\%$ improvement). This crossover qualitatively reproduces the paper’s central claim that the neural-memory mod-

ETTh1 Long-Term Forecasting — Titans LMM vs. Baselines

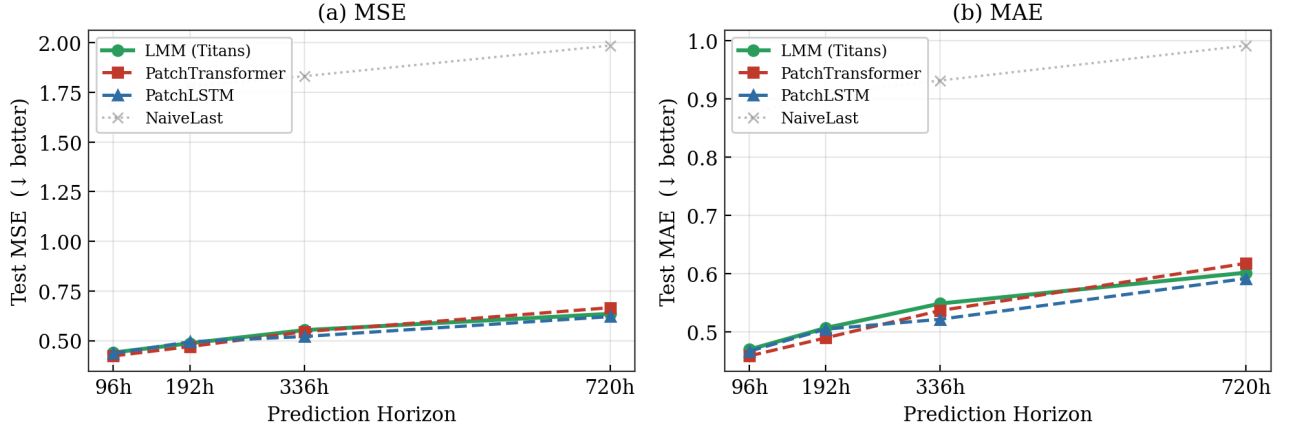


Figure 4: Forecasting error (MSE, MAE) as a function of the prediction horizon for the LMM and the baselines.

Titans LMM: advantage grows with the prediction horizon (ETTh1)

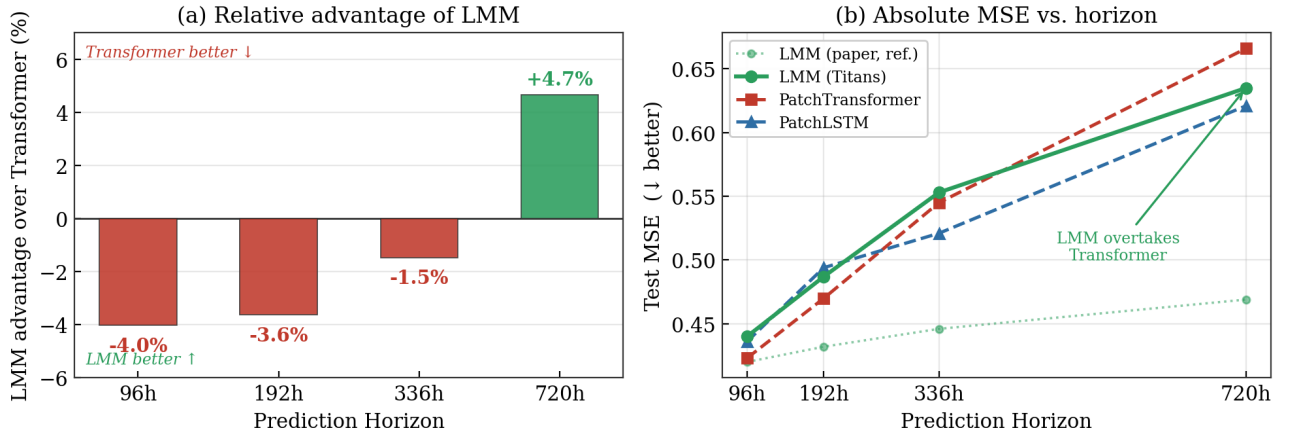


Figure 5: Relative advantage of the LMM over the Transformer. The advantage grows with the horizon and becomes positive at 720h.

ule is particularly advantageous for long-term dependencies. The same intuition appears in Experiment I: the memory holds useful information well beyond the attention window, and its relative benefit increases precisely when long-range structure matters most.

Comparison to the paper. Our absolute MSE values are 5–35% higher than those reported in the original paper (e.g. 0.420 at 96h and 0.469 at 720h). We attribute this gap to model size (our $\approx 580\text{K}$ – 3.1M parameters vs. $\approx 170\text{M}$ in the paper), the smaller number of training epochs, and the omission of Simba-specific components (depth-wise convolution, gating). Crucially, the *qualitative* pattern is preserved: the LMM wins at long horizons. This is also consistent with *Titans Revisited*, which reports that the memory-only LMM variant matches or exceeds attention-based models on forecasting tasks.

5 Summary of Results

Taken together, our tracks delineate the regime in which the Titans architecture can be reproduced under free-tier compute. The reasoning capability, which depends on large-scale pretraining of the cross-segment memory, **cannot** be reproduced at our scale — a negative result that is itself informative and corroborated by independent work. The forecasting capability, which relies on the memory module compressing temporal structure, **can** be reproduced qualitatively, with the LMM demonstrating the expected long-horizon advantage over a parameter-matched Transformer.

6 Conclusion

Our experiments delineate the regime in which the Titans architecture can be reproduced under free-tier compute. On **retrieval** (Experiment I) the MAC Titan clearly extends useful memory beyond the attention window, holding the needle for up to four times its training context where the Transformer fails. On **reasoning** (Experiment II) we obtained a negative result: the multi-hop capability depends on large-scale pretraining of the cross-segment memory and cannot be reproduced at our scale — a finding that is itself informative and corroborated by independent work. On **forecasting** (Experiment III) the memory-only LMM qualitatively reproduces the paper’s central claim, demonstrating the expected long-horizon advantage over a parameter-matched

Transformer. Taken together, the three experiments give an intuitive, hands-on account of *what* the Neural Memory stores and *when* it helps: it compresses long-range structure into its weights, and its benefit is greatest exactly when information must survive far beyond the attention window. A natural extension, given more compute, would be to pretrain the MAC on substantially larger contexts and token budgets, which we expect would close the gap on the reasoning task while further widening the Titan’s advantage on long-context retrieval and forecasting.